



Multiview Stable Diffusion Conditioned Image Editing

Candidate Number: 1088469

Word Count: 7378

University of Oxford

A thesis submitted for the degree of

Master of Science in Advanced Computer Science

Trinity 2025

Contents

1	Introduction and Motivation	4
2	Model Architecture and Attention	6
2.1	UNet and AR	6
2.2	AR within MVDream	8
3	Attention Hijacking, Masking, and Interpretability	9
3.1	MasaCtrl and Masking	9
3.2	Free Prompt Editing and Attention Hijacking	12
3.3	Visualization of Query Matrix and Cross Attention	16
4	Generalizations to MVAdapter and ControlNet	17
4.1	MVAdapter Results	17
4.2	Batch Size and Efficiency Experiments	22
5	Stable Diffusion Versions and Identity Preservation (IP Adapter)	23
6	Conclusion, Limitations, and Further Work	27
6.1	Memory Efficiency and Runtimes	27
6.2	Towards fine-tuning based methods	29
6.2.1	Parameter-Efficient Fine-Tuning (DreamBooth with LoRA)	29
6.2.2	Full Fine-Tuning	30
6.2.3	Other Directions in the Field	31
6.3	Potential Applications	31
6.4	Conclusion and Acknowledgments	32

References	33
Appendix A Reproducibility	35

Multiview Stable Diffusion Conditioned Image Editing

Candidate Number: 1088469

Abstract

Stable Diffusion is a well known model to generate images via text prompts or other input images. However, there are limitations to the accuracy of the generations if we expand the scope to incorporate multiple views of the same object for a singular prompt. Similarly, image editing is a challenge since there is a trade-off between preserving key identity characteristics of the original image and enabling large structural changes.

We introduce Attention Rewiring (AR), a tune-free modification to the self-attention mechanism within Stable Diffusion adapters aimed at improving multiview consistency. While the initial AR implementation was developed by Anjun Hu, my contribution focused on integrating, evaluating, and improving its application within multiview diffusion settings. Empirically, AR demonstrates improved spatial awareness across views, comparable to the “inflated 3D attention” mechanisms seen in MVDream and related multiview video synthesis work. Our results show that AR is particularly effective when applied to MVDream models based on Stable Diffusion 2.1 and 1.5, although results on other adapter architectures remain inconclusive.

The downstream applications of this work are evident. Clearer multiview outputs enable more accurate 3D keypoint extraction, which in turn improves mesh editing. This is useful for 3D modeling and animation pipelines, game development, and virtual reality where consistent geometry and texture are critical.

1 Introduction and Motivation

Stable Diffusion is a generative model based on denoising diffusion probabilistic processes. It synthesizes high quality images by gradually transforming a noisy latent vector into a coherent image across N timesteps. At each step, a neural network predicts the denoised version of the current latent, progressively adding semantic structure and detail [9]. This process is conditioned on input text prompts or other modalities, enabling fine-grained control over the generated content.

The model operates in a learned latent space, where the dimensionality and semantics of the latent vectors capture high-level features of the output image. Image generation typically begins from a latent sampled from a standard Gaussian distribution. However, it is also possible to reverse the generation process and estimate the latent that would produce a given real image. One such technique developed is called DDIM Inversion, which is a deterministic process [10].



Figure 1: Inverting and Articulating Multiview Tiger Latents

DDIM Inversion unlocks a variety of powerful editing workflows. By recovering the latent of an input image, we can articulate it into new forms by altering pose, shape, or structure while still preserving style and identity. In this context, "articulation" refers to the forward

generation process starting from an existing latent, guided by modified prompts or structural constraints. This makes it possible to produce consistent multiview outputs, style-preserving edits, and fine-tuned reconstructions that would not be possible through naive prompting alone.

While initializing with partially denoised image latents is a strong starting point, it has become a common baseline. Many followup works have introduced their own modifications to improve editing quality. In practice, this step is often necessary but not sufficient for producing coherent and faithful image edits.

So far, this applies primarily to single-view editing. In order to extend to multiview, camera embeddings are injected along with each timestep in the UNet, along with the intrinsic and extrinsic camera angles. This gives the model awareness of the rotational angle used to generate the different views.



Figure 2: Problems with Multiview Generations [1]

However, when extending to multiview image generation, new challenges emerge. These include issues such as the multiview Janus problem, where different views resemble different identities, and content drifting, where semantic attributes shift inconsistently across views. To address these issues, works such as MVDream use Spatial (3D) Self attention to address the multiview consistency. In their paper, they said: "Specifically, given a tensor of shape $B \times F \times H \times W \times C$, we format it as $B \times FHW \times C$ for self-attention (Batch, Frames, Height, Width, and Channel respectively), where the second dimension is the sequence dimension representing the tokens [1]. In such a way, we could also inherit all the module weights from original 2D self-attention." However, while this method gives more spatial awareness to the model, it's far from completely solving this issue.

Our proposed method, Attention Rewiring, addresses this challenge by modifying the struc-

ture of the attention mechanism itself. Unlike adapter-based approaches, this is a tune-free intervention so no additional parameters are introduced and the model remains frozen during inference. Instead, we restructure the self attention in a way that better aligns with multi-view consistency, similar in spirit to how MVDream made its modifications. This allows for improved reasoning across views.

2 Model Architecture and Attention

2.1 UNet and AR

The fundamental architecture that Stable Diffusion is built on is the UNet, where a series of encoders and decoders lower the dimensionality of the information before up scaling it back to the original size. In the middle, a series of residual connections make sure that not too much of the source information is lost.

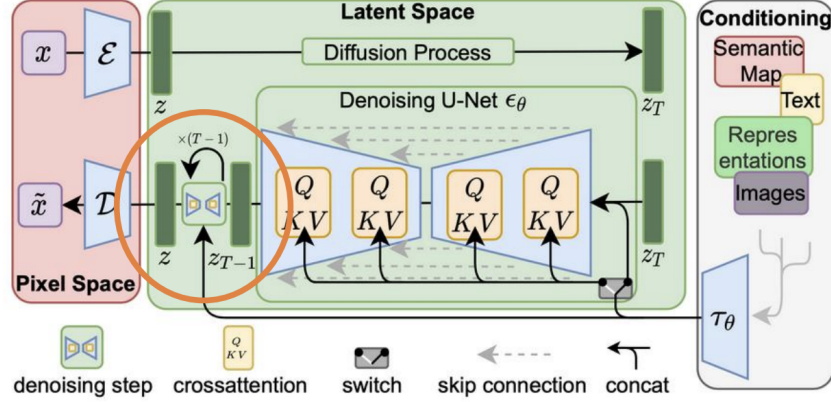


Figure 3: Architecture of Latent Stable Diffusion [9]

The specific attention mechanism is defined as the following:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

However, to fully understand AR, we must first introduce the concept of the original input

image vs the current generated images. X_0 refers to the cached latent trajectory recovered by DDIM inversion of the original input image, which remains fixed throughout generation. X_1 refers to the newly generated images. So, while X_1 is being generated by new "AR" attention, X_0 will always return to the original input image.



Figure 4: Interleaved multiview generation

With that being said, **Attention Rewiring (AR)** is illustrated in Figure 5. Each input is first split by view, separating into X_0 and X_1 . The X_0 branch is exclusively processed via *self-attention*, where it attends only to itself. Meanwhile, AR modifies the self-attention pathway of X_1 to improve multiview consistency [11].

For each view index $i \in \{0, \dots, N\}$, a unique composition of views is formed by padding $X_{0,i}$ with repeated copies of X_1 for all other views, resulting in a new tensor that combines cross-view information. *Self-attention is then computed* using:

- **Query:** from X_1
- **Key/Value:** from the padded tensor $[X_{0,i}, X_1, \dots]$

For example, when processing view 0, the query would be X_1 , and the key and value would be $X_{0,0}, X_{1,1}, X_{1,2}, \dots, X_{1,N}$. When processing view 1, the query would be X_1 , and the key and value would be $X_{1,0}, X_{0,1}, X_{1,2}, \dots, X_{1,N}$. In essence, the view i uses X_0 as guidance for itself for what it "should" look like, while also attuning to the other views to ensure consistency. By combining the information, we ensure the i th view of X_1 doesn't deviate too far from the source information. This produces N different outputs, one per view. For each view i , only the i -th output is *plucked* from the attention result. These are concatenated to reconstruct the final view-aware version of X_1 , which we call **Combined X_1** .

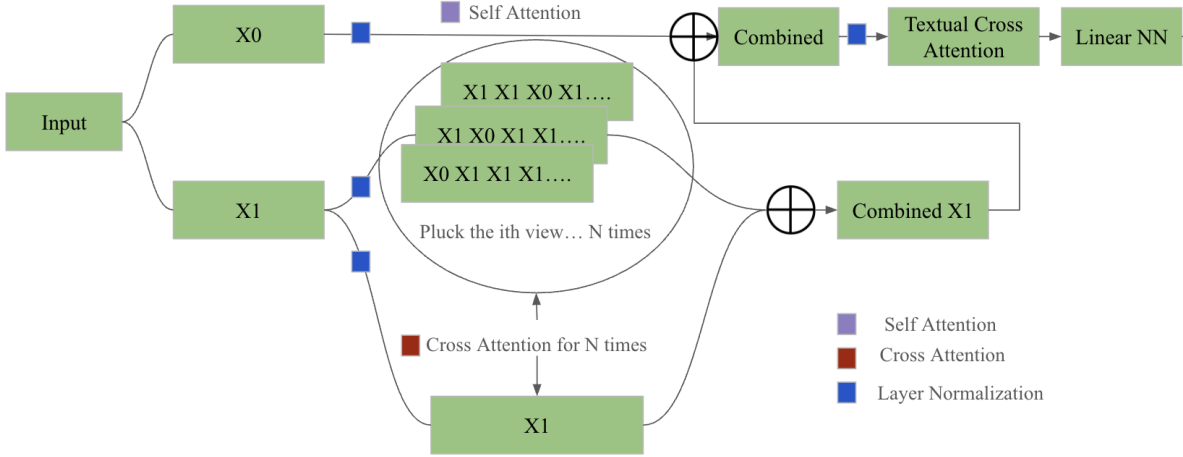


Figure 5: AR Architecture

Throughout the process, *layer normalization* is applied at each key step. The outputs from both paths— X_0 's cross-attention and the rewired self-attention of X_1 —are then merged and passed through a **Textual Cross Attention** where it attends to the embedded text features (e.g., CLIP prompt embeddings). A **final linear layer** produces the output.

2.2 AR within MVDream

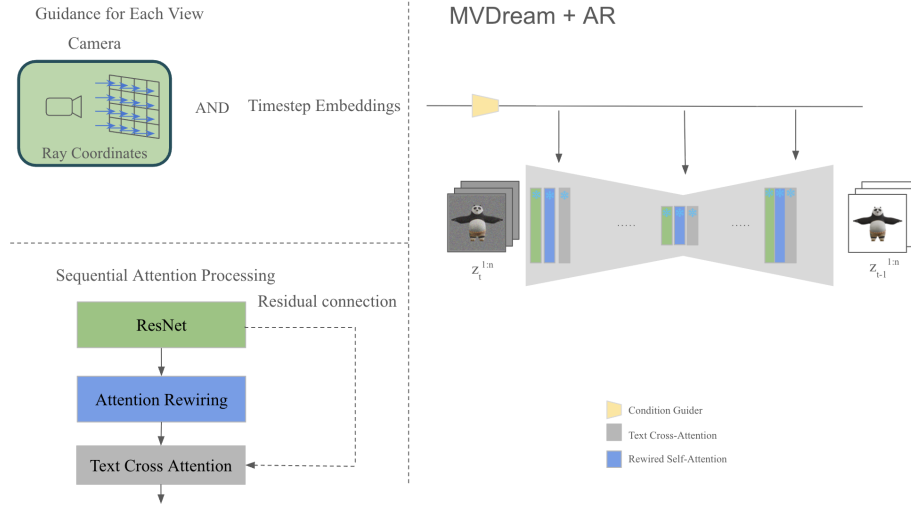


Figure 6: MVDream and AR architecture. Figure adapted from Oishi Deb [11], with modifications by the author.

As a reminder, MVDream is a multiview model finetuned from Stable Diffusion 1.5 and 2.1. Figure 6 is the architecture of MVDream, with the self attention mechanism being fully replaced with AR. At each timestep, the camera embeddings are injected along with the timesteps into the UNet. The attention is processed sequentially, first going through a ResNet, then AR, then Textual Cross Attention along with residual connections. Most examples are generated with MVDream on Stable Diffusion 2.1, unless stated otherwise.

Figure 7 is a baseline comparison between multiview edited images generated with the default self attention in MVDream using partially denoised latents, and using AR. The posture is correctly generated of the tiger sitting down, and there are no issues of the overall structure. Although AR does improve the generation, there are some consistency issues with the tiger example, such as missing a tail on the first frame and losing the identity of the tiger to some extent as the color is a different shade of orange.

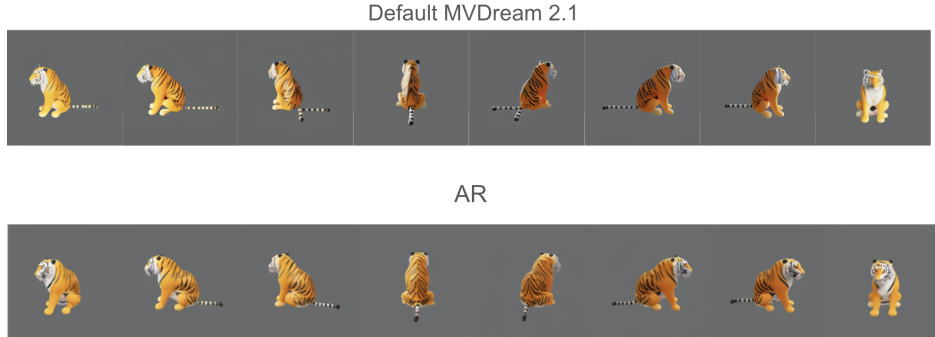


Figure 7: Effects of AR

The question now remains, how is it possible to improve AR in the context of MVDream?

3 Attention Hijacking, Masking, and Interpretability

3.1 MasaCtrl and Masking

The idea of masking is long-standing in computer vision. MasaCtrl, an image editing paper, was the first to propose the idea of masking the query matrix during the self-attention block at each timestep in order to separate the background and the foreground. By using a binary

threshold, objects can only attune to themselves and not the background which may contain similar colors [12].

In their context, they create a parallel generation with the original source prompt. As the articulation progresses through each timestep, it creates a binary mask using the softmaxed attention values of the cross attention. The cross attention mask for the source prompt is then averaged with the mask of the target prompt for the final mask used for the articulation process. We similarly follow this idea, and process both articulations concurrently, using a threshold of 0.63 determined experimentally for the tiger specifically.

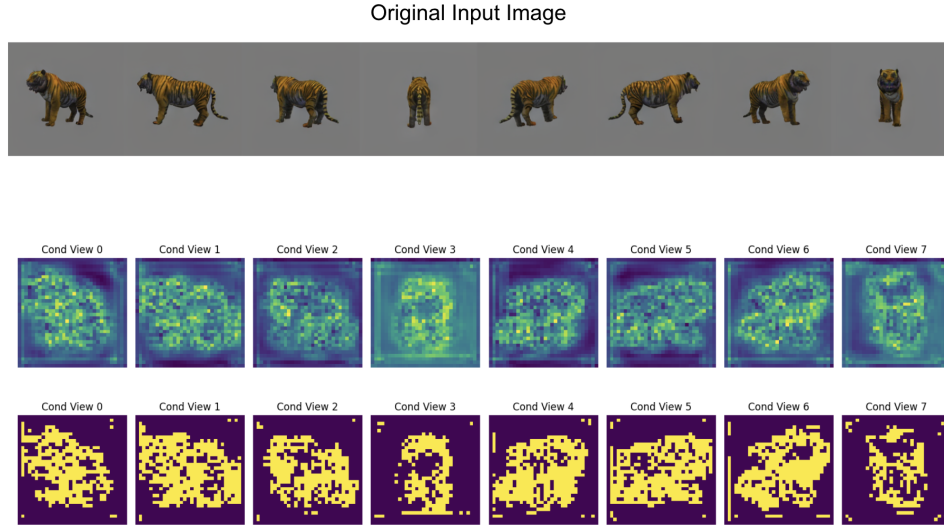


Figure 8: Extracted Binary Masks at Layer 0

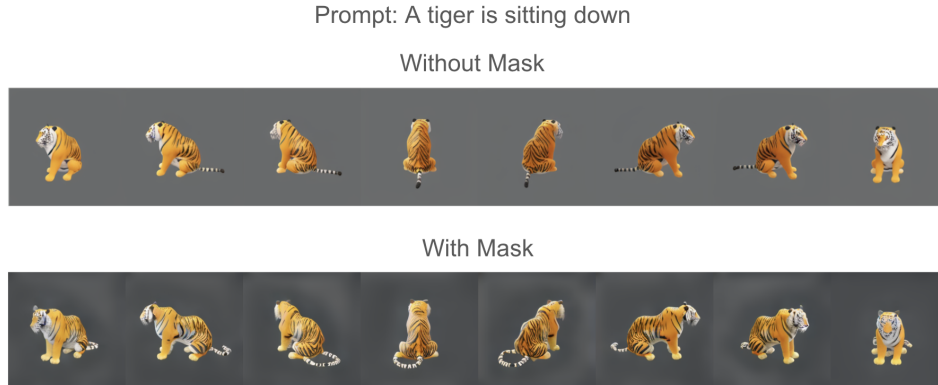


Figure 9: Effects of Cross Attention Mask

To formally define the mask, let $\mathbf{Q}^{\text{src}} \in \mathbb{R}^{N \times d_k}$ be the query matrix from the *source* generation (one row per spatial position), and let $\mathbf{K}^{\text{text}} \in \mathbb{R}^{T \times d_k}$ be the text-key matrix from cross-attention (e.g., CLIP embeddings, typically $T=77$ tokens). Define the source query-text score matrix by

$$\mathbf{S}_{\text{src}} = \frac{\mathbf{Q}^{\text{src}}(\mathbf{K}^{\text{text}})^\top}{\sqrt{d_k}} \in \mathbb{R}^{N \times T}.$$

Apply a softmax over the token dimension to obtain attention weights $\mathbf{A}_{\text{src}} \in \mathbb{R}^{N \times T}$:

$$\mathbf{A}_{\text{src}}^{ij} = \frac{\exp(\mathbf{S}_{\text{src}}^{ij})}{\sum_{j'=1}^T \exp(\mathbf{S}_{\text{src}}^{ij'})}.$$

Aggregate over tokens to get a foreground score per spatial position (e.g., a mean over tokens):

$$p_i = \frac{1}{T} \sum_{j=1}^T \mathbf{A}_{ij}, \quad i = 1, \dots, N.$$

Threshold to obtain a binary mask $\mathbf{M} \in \{0, 1\}^N$:

$$\mathbf{M}_i = \begin{cases} 1, & p_i \geq \tau, \\ 0, & \text{otherwise.} \end{cases}$$

Broadcast $\mathbf{M}$ over the key dimension to form $\tilde{\mathbf{M}} \in \{0, 1\}^{N \times N}$ (and over heads in practice), and apply it *after* softmax to the current self-attention:

$$\text{Attn}_{\text{masked}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = (\tilde{\mathbf{M}} \odot \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)) \mathbf{V}.$$

where $\odot$ denotes element-wise multiplication, d_k is the key dimensionality, and τ is the empirically chosen threshold (e.g., $\tau = 0.63$ for the tiger example). This ensures that foreground regions, as determined from the source queries, can only attend to themselves and not to background regions with similar colors. The same is repeated for the mask using the text and query from the current, target generation, and the masks are averaged.

In Figure 9, the addition of the mask clearly preserves the structure and posture of the

tiger more faithfully. In particular, the back shows a distinct arch that defines the hips, and previously missing parts such as the tail are successfully reconstructed. Fine-grained details, particularly in the facial structure and the striping pattern, are also more accurately rendered. The camera angles remain consistent across views, confirming that the articulation process is not disrupted by the introduction of the mask.

Although this parallel generation strategy incurs additional computational cost during inference, due to the need to maintain and query against an auxiliary generation stream, it significantly improves spatial coherence and semantic consistency. The model benefits from the ability to attend to a temporally aligned but spatially disentangled representation, which appears to mitigate common issues such as texture drift, mode collapse, or foreground-background confusion. In particular, the binary mask enforces attention locality by forcing each region to selectively attend to relevant foreground features, enabling sharper structural fidelity across frames.

3.2 Free Prompt Editing and Attention Hijacking

One issue remains with the tiger generation. The coloring of the tiger is rather different from the input source, showing that although the pose and semantics are correct, the identity is slightly drifting from the input. This is an issue because common keypoint detection algorithms, like SIFT, include color. In order to generate better keypoints for the iterative 3D optimization, the identity needs to be preserved better.

Free prompt editing (FPE) is a paper that suggests an improvement to this issue, by "hijacking the attention" at certain layers and timesteps [4]. Inside each UNet, 50 timesteps of denoising are run. At each timestep, there are 16 layers of the autoencoder. Instead of querying the parallel generation's key and query value, "attention hijacking" replaces the self attention of the current generation with the self attention of the parallel generation. Of course, this must be done at earlier timesteps and layers, otherwise there wouldn't be enough time for it to converge on the new posture. Empirically, FPE found that layers 4 - 14 is the optimal range

for this hijacking. We integrate this idea with AR, and in addition to the masking that is generated.

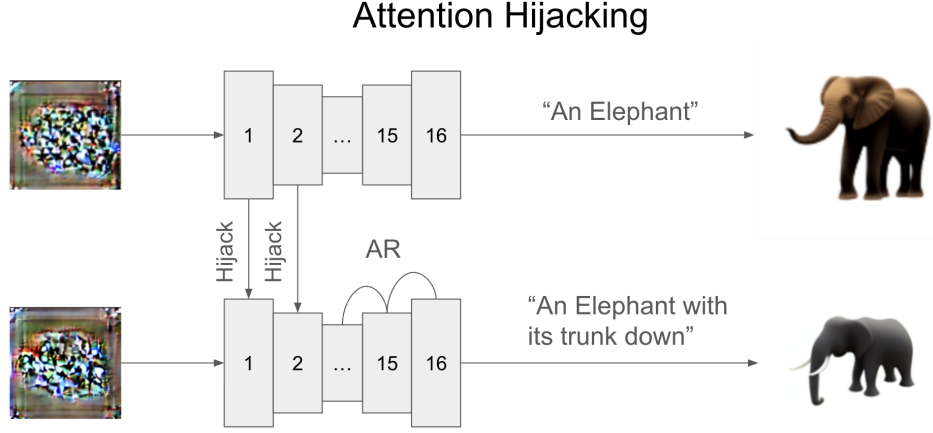


Figure 10: Design of Attention Hijacking

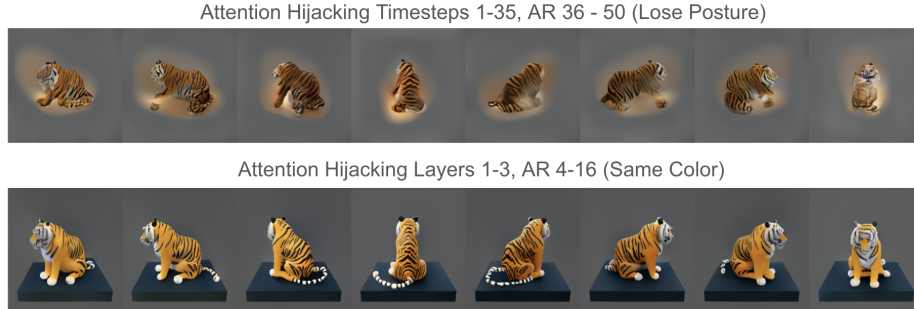


Figure 11: Different FPE ablations

If we use too many steps of FPE, we can see that the posture and structural identity is degraded, and if we use too little FPE, the color remains shifted. In order to discover the optimal hyperparameters for our experiments, an ablation was ran to empirically determine the results. As such, both layers and timesteps needed to be considered. For each layer L and timestep T , let $\mathbf{Q}_{L,T}$, $\mathbf{K}_{L,T}$, and $\mathbf{V}_{L,T}$ denote the query, key, and value matrices respectively.

We define the self-attention output at layer L and timestep T as:

$$\text{SA}_{L,T} = \begin{cases} \text{Attention}(\mathbf{Q}_{L,T}^{\text{src}}, \mathbf{K}_{L,T}^{\text{src}}, \mathbf{V}_{L,T}^{\text{current}}), & \text{if } L, T \in \mathcal{A} \\ \text{Attention}(\mathbf{Q}_{L,T}^{\text{current}}, \mathbf{K}_{L,T}^{\text{current}}, \mathbf{V}_{L,T}^{\text{current}}), & \text{otherwise} \end{cases}$$

where $\mathcal{A}$ denotes the set of layers and timesteps selected for attention hijacking (e.g., layers 4–14 as found by FPE), and

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \mathbf{V}$$

is the standard scaled dot-product self-attention. The difference here is the Query and Key matrices are from the source, but the Values are taken from the current generation. To systematically evaluate the impact of hijacking across timesteps and layers, we adopt a two-phase flag design. The generation process is split into two segments: an *early-stage 3-block design* and a *late-stage 2-block design*, separated by a predefined timestep (e.g., $T = 35$).

In the first phase ($T = 0$ to $T = \text{Switch}$), the architecture follows the sequence:

$$\text{AR} \rightarrow \text{FPE} \rightarrow \text{AR}$$

This setup ensures that attention hijacking occurs in the middle block (FPE), while the initial AR layer encourages stable posture convergence and coarse multiview alignment early on. The transition layers between $\text{AR} \rightarrow \text{FPE}$ and $\text{FPE} \rightarrow \text{AR}$ are ablation hyperparameters as well. In the second phase ($T > \text{Switch}$), the architecture simplifies to:

$$\text{FPE} \rightarrow \text{AR}$$

Here, hijacked attention continues through FPE, but with fewer processing blocks, enabling finer-grained refinements without disrupting structural stability already achieved. Once again, the layer going from $\text{FPE} \rightarrow \text{AR}$ is a hyperparameter. This layered, time-dependent switching

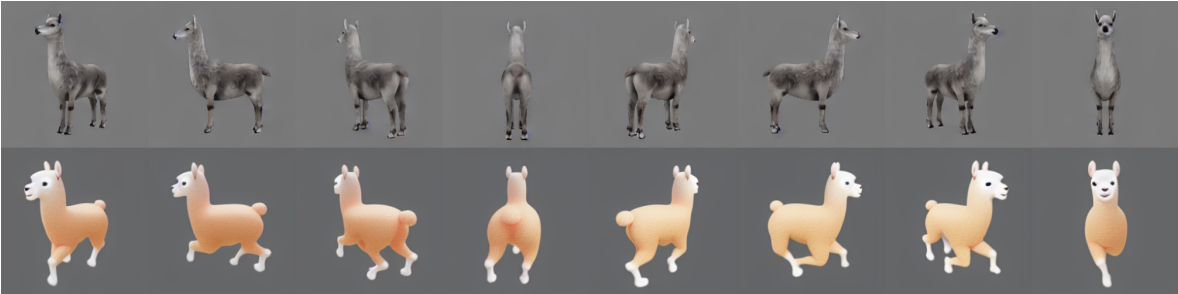
strategy enables controlled manipulation of self-attention without sacrificing global consistency or requiring additional training.

Following an ablation study, we observe that the proposed method generally improves results, though its impact may vary across images. For the tiger example specifically, the best configuration was found to be: applying FPE to all layers for the first 15 timesteps, followed by 35 timesteps where FPE is applied only to the first 5 layers, with the remaining layers using AR. This hybrid setup results in a significantly darker appearance that better matches the original input image, while preserving structural integrity and posture, effectively balancing fidelity and alignment.



Figure 12: Final result of tiger sitting down, with FPE

Unfortunately, this idea works much better with single color than it does with multi-color images, such as the hummingbird. It also struggles with other animals, such as the alpaca. The following images compare the alpaca edit with and without the presence of FPE.



(a) Input and generation with mask (0.4), AR, and no FPE



(b) Generation with mask (0.4), AR, and FPE

Figure 13: Comparison of alpaca generations. Prompt: "An alpaca is running."

As we can see, attention hijacking is very sensitive to hyperparameters and might not be suitable for all generations. In suitable cases, it drastically improves the fidelity of the generated object.

3.3 Visualization of Query Matrix and Cross Attention

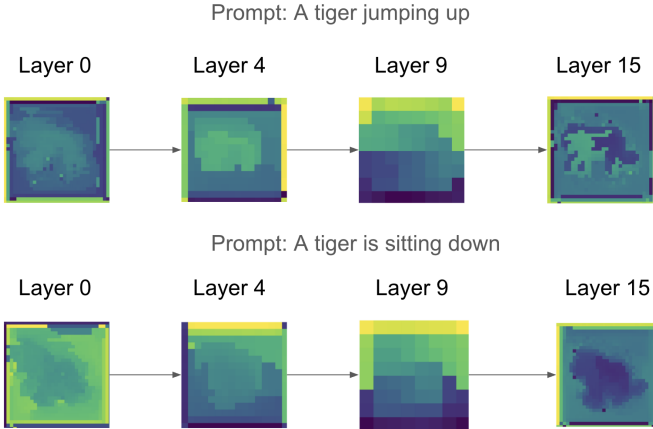


Figure 14: Different Layers Within One Timestep

Interpretability is a critical component of modern machine learning, especially in the context of diffusion models. To better understand the behavior of our masking and attention hijacking mechanisms, we visualize the query matrices across different layers and timesteps during generation. Inspired by the probing methodology used in MasaCtrl, we apply UMAP to reduce the dimensionality of the queries and visualize their progression across layers [12].

In the example shown in Figure 14 at timestep 15, clear differences emerge between two semantic conditions: a tiger sitting versus a tiger jumping. Interestingly, query representations begin to diverge meaningfully from layer 14, aligning with MasaCtrl’s findings that semantic information, such as pose, color, and context, is encoded deeper in the network. This validates our decision to perform attention hijacking starting from layer 4, where such features are more pronounced. This is why the first half the ablation is split into 3 blocks, to test the validity of this layering. However, for the tiger case, we empirically found that sandwiching with AR was often redundant, the FPE block alone was sufficient to steer the semantics.

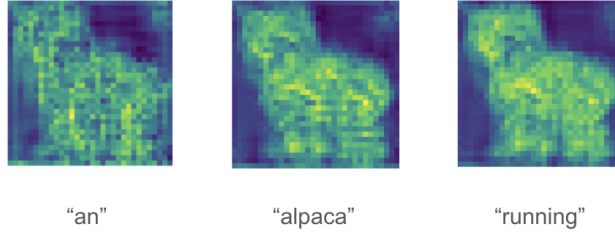


Figure 15: 90 Degree View Alpaca Heatmap. Layer: 0, Timestep: 15

When the tokens are split apart in the cross context visualizations, we are able to see the contributions of each individual word. However, since CLIP is a transformer based, the relative context of each word in the sentence is carried over in the encoding. Our experiments support this, unlike the visualizations in MasaCtrl, in which each word seems to lack the context of other tokens. One possible explanation for the difference is that the visualizations may have been ran individually for each word in MasaCtrl instead of together [12].

4 Generalizations to MVAdapter and ControlNet

4.1 MVAdapter Results

One natural direction for improving generation quality is to enhance the baseline model on which Attention Rewiring is applied. Earlier sections focused on MVDream, a multiview diffusion model fine-tuned from Stable Diffusion 2.1. However, a more recent and modular alternative, MVAdapter, has since been released. MVAdapter is a compact (approximately 20 million parameter) adapter designed to operate on top of Stable Diffusion 1.5, 2.1, or SDXL without requiring full model fine-tuning. Unless otherwise stated, all experiments in this section use MVAdapter with Stable Diffusion 2.1 as the default configuration [2].

To effectively adapt AR to MVAdapter, it is crucial to first understand MVAdapter’s architectural differences. Its core innovation lies in the introduction of multiview (MV) blocks, which promote consistency across different camera viewpoints. These blocks use cross-attention to query from sibling views during the same generation step, similar in spirit to AR. However,

unlike AR, MVAdapter does not rely on cached “source” generations. Instead, it attends only to other camera angles, either at the same elevation (e.g., 0°) or at the same azimuthal positions, in order to maintain geometric coherence.

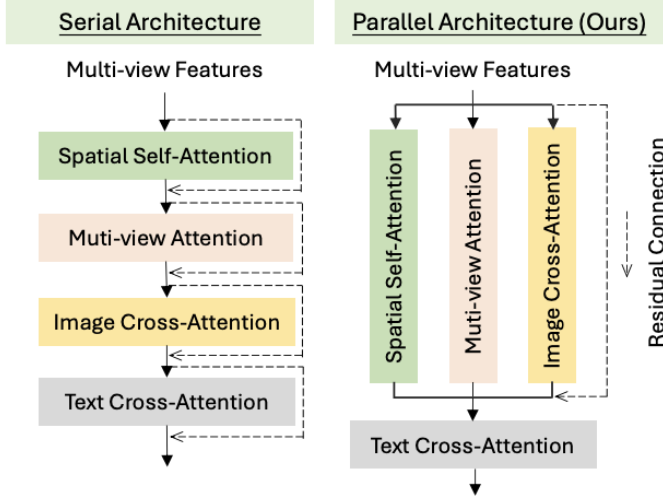
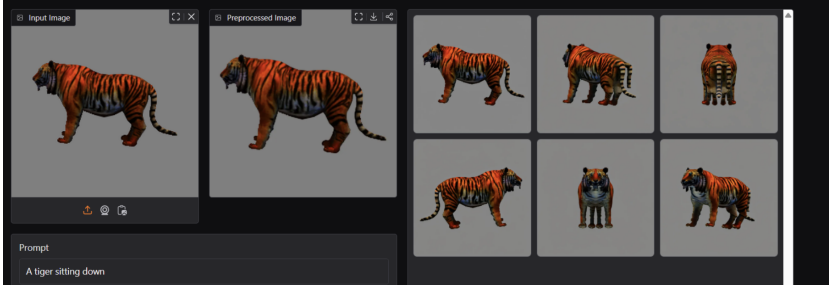


Figure 16: MVAdapter architecture illustration [2].

A significant distinction between MVAdapter and MVDream lies in attention scheduling. MVDream processes attention sequentially, applying a ResNet block, followed by self-attention (or AR), and then text cross-attention. MVAdapter, by contrast, parallelizes all operations except for textual cross-attention. This leads to improved inference efficiency. MVAdapter also includes an image reference module, allowing external reference images to guide generation—useful for identity or appearance preservation.



(a) Prompt: "A tiger sitting down"

Figure 17: Baseline generation using MVAdapter with image reference only.

To test its editing capabilities, we used the reference image of a tiger alongside a target

prompt "a tiger sitting down". However, the resulting generation revealed that reference-based guidance alone was insufficient for structural edits. This behavior aligns with findings from MVDream: latent inversion, specifically DDIM inversion, is required to preserve structural consistency and enable meaningful edits.

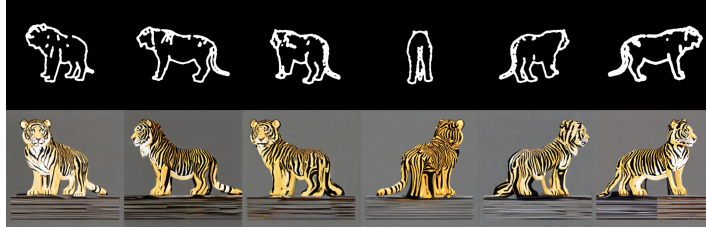


Figure 18: Scribble-based ControlNet conditioning.

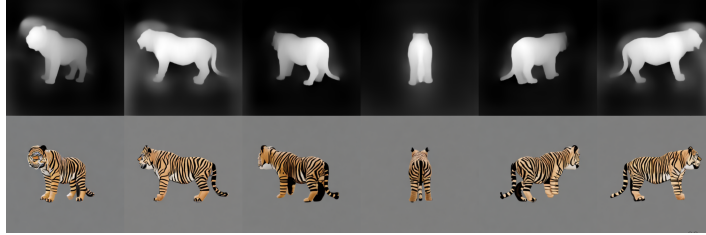


Figure 19: Depth-based ControlNet conditioning.

Before applying AR, we first explored the use of ControlNet as a conditioning mechanism. ControlNet augments diffusion models by introducing an auxiliary network that injects control signals (e.g., edge maps, depth, scribbles) into intermediate layers of the model. This enables fine-grained, controlled generation while preserving the underlying behavior of the base model. While pose estimation is conceptually promising for posture control, it failed in our tests involving animals, as existing ControlNet checkpoints are primarily trained on humans [7]. Consequently, we reintroduced DDIM inversion and incorporated AR to re-establish structural consistency.

This led to a key question: how should AR be integrated into MVAdapter? Should it replace the existing self-attention layers, or be added in parallel? Initial experiments where AR was added as a parallel branch resulted in failure to converge. We first hypothesized this was due to improper normalization, but even after addressing those, the convergence issue persisted.



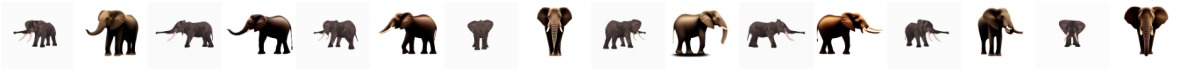
Figure 20: Cannot Converge under Global Scaling of 2/3

Interestingly, convergence could be recovered by scaling the branches individually. For instance, when using weights of 1.0 for self-attention, 0.5 for MV blocks, and 0.5 for AR, generation succeeded. In contrast, a uniform scaling (e.g., 1.0 for all, followed by global 2/3 normalization) resulted in non-converging generations. This suggests that AR and MV blocks, while architecturally similar, may introduce redundant signals when combined at full strength. AR uses cached latents from previous timesteps (source generation), while MV blocks attend to sibling views at the current timestep. Although conceptually distinct, both inject global view-consistent signals into the model, potentially over saturating attention dynamics when applied in parallel.

Our findings suggest that AR and MV blocks are not orthogonal—they encode overlapping forms of inter-view consistency. When applied individually, each enhances generation quality, but when combined, they require careful balancing to prevent instability.



(a) A tiger is sitting down.



(b) Elephant with its trunk down.



(c) A hummingbird is sitting.

Figure 21: MVAdapter generations for non-human animals using 0.5 Multiview, 0.5 AR.

While MVAdapter demonstrates significantly improved identity preservation compared to MVDream, its ability to follow *camera embeddings* and *posture guidance* begins to degrade under certain conditions. Specifically, the central views of the *elephant* and *hummingbird* should depict subjects facing away, but generations often fail to reflect this, for example, the elephant’s trunk is often misplaced. It is important to note, however, that MVAdapter

was originally trained on only **six camera views**, not eight. As a result, extreme or interpolated angles such as 135° and 225° fall outside the model’s training distribution, making generalization to these perspectives more difficult.

Another interesting experiment involves completely removing the multiview block while retaining the other parallel components, including AR. This setup was designed to probe how the model behaves when deprived of explicit inter-view consistency mechanisms. While the tiger appears to converge visually, its posture is noticeably degraded, and the coherence across different views is almost entirely lost.

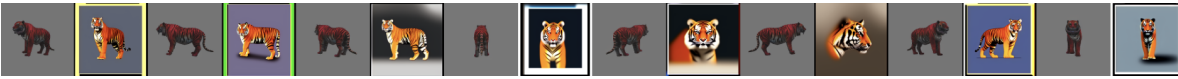


Figure 22: Generation without the multiview block, using only the AR attention module.

These results suggest that the multiview block plays a crucial role in maintaining geometric consistency across camera angles. In contrast, AR appears to contribute primarily to local spatial refinement and depth cues, helping avoid overly flat or structurally ambiguous generations. The two components thus serve complementary roles in enforcing both global and local structure.

Ultimately, we acknowledge that the broader line of experimentation of integrating AR as a parallel block within MVAdapter did not yield consistently successful results. While MVAdapter offers a modular and lightweight alternative to MVDream, our attempts to combine it with AR in a parallel attention setup led to convergence issues, redundancy between modules, and degradation in spatial and structural consistency. Efforts to balance AR and multiview blocks through scaling helped partially but did not result in robust or interpretable improvements across tasks.

In particular, the removal of the multiview block confirmed its importance in maintaining coherent multiview geometry. Without it, even though the generation may converge in appearance, posture fidelity and cross-view consistency break down. In comparison, MVDream, despite being larger and less modular, continues to outperform MVAdapter.

However, the small parameter footprint of the MVAdapter (approximately 20 million parameters) makes it an appealing candidate for future fine-tuning. Its lightweight design offers a more feasible path to domain-specific or low-resource adaptation, especially compared to the computational demands of fine-tuning full diffusion backbones. Further work is needed to better understand the optimal integration of AR into such architectures, potentially through targeted architectural tuning or joint training.

4.2 Batch Size and Efficiency Experiments

`F.scaled_dot_product_attention` from PyTorch’s package is a notable advantage of MVAdapter’s codebase in place of traditional `einsum` operations. This change significantly improves computational efficiency in attention computation. However, for the purpose of our ablation experiments, this alone was insufficient—so we integrated xFormers, a memory-efficient attention library, to further optimize performance. This modification resulted in a substantial speed-up: the runtime for a single generation dropped from approximately 15 minutes to just 5 minutes.



Figure 23: Sanity check with original attention implementation.



Figure 24: Sanity check with xFormers attention. Identical outputs, improved efficiency.

As shown in Figures 23 and 24, both attention implementations produce consistent outputs, confirming correctness. However, xFormers achieves this with significantly improved runtime efficiency. What proved particularly interesting was the impact of batch size on runtime during different stages of the pipeline. Recall from Figure 5 that X_1 is processed N times—once per view—and each instance is paired with the corresponding X_0 view to guide the generation. By default, this results in a batch size of 2 per step. To explore performance scaling, we experimented with various batch sizes. As expected, increasing the batch size to $2N$ (process-

ing all view pairs in a single batch) resulted in the fastest articulation runtime, while smaller batch sizes were significantly slower.

Surprisingly, the opposite trend held during DDIM Inversion: smaller batch sizes led to faster inversion, while large batches degraded performance. This inversion behavior highlights a key distinction in how different components of the pipeline utilize memory and parallelization, and suggests that optimal batching strategies may need to be task-specific. During articulation, generation involves forward diffusion passes with highly parallel attention operations (e.g., AR or MV blocks). Larger batch sizes enable more efficient GPU utilization by maximizing parallel computation, reducing loop overhead, and taking advantage of optimized memory access patterns. In contrast, DDIM inversion requires many sequential denoising steps per sample. Larger batches significantly increase memory pressure, as activations and intermediate states must be maintained across all timesteps. When memory limits are exceeded, this leads to paging or degraded memory bandwidth, ultimately slowing down performance. The contrasting performance behavior between articulation and DDIM inversion might be attributed to how each process utilizes memory and parallelism.

5 Stable Diffusion Versions and Identity Preservation (IP Adapter)

Another strategy we explored for preserving identity in MVDream generations was the use of additional conditioning methods, specifically IP-Adapter. Similar to ControlNet, IP-Adapter is a frozen module that encodes the reference image into an embedding, which is then concatenated with the textual context during generation. This allows the model to retain source-specific visual information—such as identity and appearance—at every step where the image is conditioned on the prompt. Recall that MVDream’s primary shortcoming lies in identity and color fidelity, not posture. While other ControlNets excel at preserving geometric properties like depth or pose, they are not suited for tasks requiring fine-grained identity control.

In contrast, IP-Adapter is explicitly designed for this purpose. However, a key limitation arises: IP-Adapter is only compatible with Stable Diffusion 1.5 and Stable Diffusion XL,

whereas MVDream was primarily trained on Stable Diffusion 2.1 [6]. Although a version of MVDream based on SD 1.5 is available on Hugging Face, we found that the quality of its generations is noticeably degraded compared to its SD 2.1 counterpart. The generated images are below at 25 and 26.



Figure 25: Default MVDream 1.5 Generation



Figure 26: Default MVDream 2.1 Generation

In contrast, the Figures 27, 28 show results from the default MVDream. When cached latent and image-editing techniques are applied to MVDream 1.5, the model shows minimal changes in posture. It also becomes more sensitive to hyperparameters such as depth and ablation seeds. In this configuration, both posture changes and convergence present challenges. In fact, neither of the following figures reflects the intended text edit of “sitting” during the edited generation. Once IP Adapter is incorporated, however, the overall generation quality improves significantly as seen in Figure 29.



Figure 27: MVDream1.5 with AR depth=25 "A tiger is sitting on the ground"

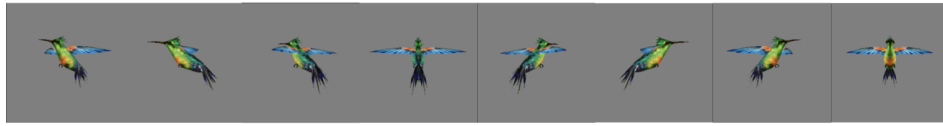


Figure 28: MVDream1.5 with AR depth=25 "A tiger sitting down"



Figure 29: MVDream1.5 with AR, and IP Adapter

The color and posture are generated correctly, with all features preserved for multiview consistency. Although the tiger appears somewhat cartoonish, it meets many of the criteria we originally aimed for. While IP Adapter demonstrates greater robustness than the default MVDream 1.5, it remains sensitive to hyperparameters. One drawback is its tendency to homogenize colors when the source image contains multiple gradients. The hummingbird example illustrates this effect clearly.



(a) Input Original Hummingbird



(b) MVDream1.5 with AR, and IP Adapter. Seed: 2025



(c) MVDream1.5 with AR, and IP Adapter. Seed: 2030

Figure 30: Prompt: "A hummingbird is sitting down"

In order to improve the generation quality even further, the masking from the previous sections were tried with IP Adapter. However, with masking, the generation results refuse to properly converge. We tried a couple iterations of masking as well, and through different thresholds. As we can clearly see, the softmaxed values are approximately normally distributed around 0.5, and we deem everything above the threshold to be "important", and therefore part of the object mask. The rest must be the background mask.

It is interesting to note that the distribution is roughly the same shape as MVDream2.1,

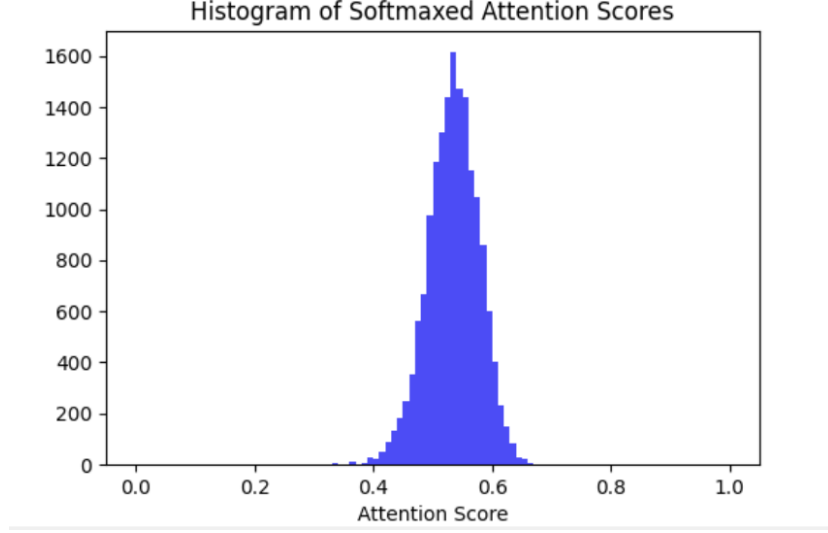


Figure 31: MVDream 1.5 - Histogram of Softmaxed Values

except the median is shifted to the left by about 0.1. So, before 0.63 was an ideal mask, and we tried 0.53 but to no avail, as well as 0.4, 0.6, and 0.7. Figure 32 is an example of the results. In fact, posture improves, but fine-grained details are lost.



Figure 32: MVDream 1.5 AR and Masking

We also tried some other versions of the masking. Instead of masking halfway through the attention computation, we tried masking AFTER the attention is fully processed, essentially zeroing out the image representation at certain spots. To define this formally, during the computation of attention for the current generation, this mask is applied *after* to the softmax normalization, but before the dot product with V . The masked attention is then given by:

$$\text{Attn}_{\text{masked}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{M} \odot \left(\text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V} \right),$$

where $\odot$ denotes element-wise multiplication, d_k is the key dimensionality, and τ is the empirically chosen threshold hyperparameter. One problem that occurs by zeroing out the image

representation is that the image is blurry and loses resolution. However, this can be fixed by letting the last couple timesteps run normal AR instead of the masked version. Unfortunately, although there were random generations of incredible quality, this masking is not generalizable to other animals and postures.

Another approach involved adding a new Color Palette ControlNet to MVDream 1.5. However, because its encoder and decoder use a different structure, the tensor dimensions did not align. The idea behind Color Palette is to compute the down sampled average color for a specified region, then encode this information and inject it into the UNet at each timestep within the early layers. Only the dimensions of the first layer were compatible, so we attempted injection there, but this corrupted the generation instead of improving it [5]. Although fine-tuning the model could resolve the mismatch, we were unable to pursue this option due to limited computational resources and time constraints.

6 Conclusion, Limitations, and Further Work

6.1 Memory Efficiency and Runtimes

Both **MVDream 2.1 + AR + Masking** and **MVDream 1.5 + AR + IP-Adapter** are capable of producing posture changes in multiview images while partially preserving source identity. However, their computational efficiency differs significantly.

Masking in MVDream 2.1 requires a parallel generation stream from the source, effectively doubling runtime. Furthermore, memory-efficient attention packages such as **xFormers** do not allow custom attention handling without sacrificing performance. As a result, masking cannot leverage the most optimized operations.

By contrast, MVDream 1.5 with IP-Adapter is substantially more efficient. IP-Adapter integrates as a simple concatenation with the CLIP context to guide cross-attention, avoiding the need for a second parallel generation. This enables inversion and generation to be completed up to **6× faster** than MVDream 2.1 with masking.

A further limitation is the inability to fine-tune within the scope of this project. With additional time and compute resources, MVDream 1.5 could be fine-tuned for compatibility with the ColorPalette ControlNet, and a custom MVDream-SDXL could be trained to integrate with the IP-Adapter SDXL version. Such improvements would likely yield substantial gains in generation quality. For context, the MVDream paper reports using **32 A100 GPUs for 3 days** to train the 2.1 variant (approximately 860M parameters), while SDXL is roughly 2.6B parameters—placing such training beyond the feasible scope of this work [1].

Finally, MVAdapter with AR remains an area of interest. Adjusting its parameters to properly balance the influence of AR and MV blocks may resolve current conflicts and prevent destabilization in generation.

The following are the results of images the original MVDream struggled to edit. The results are MVDream 1.5 with AR and IP Adapter. Even with IP Adapter, the model is struggling to edit the rabbit.

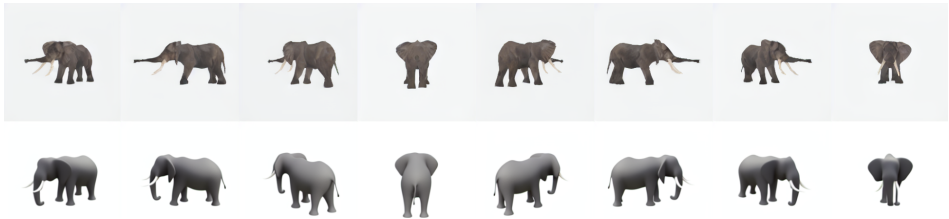


Figure 33: "An elephant lowering its trunk"

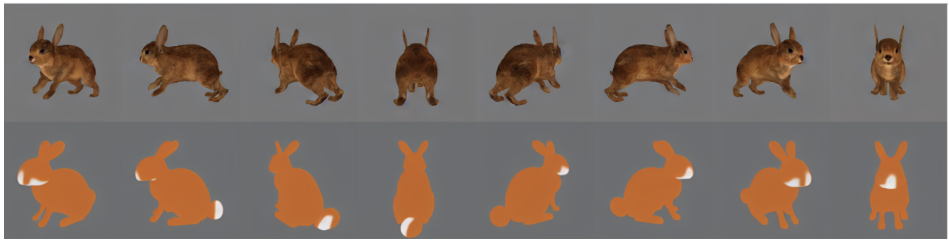


Figure 34: "A rabbit sitting down"



Figure 35: "An alpaca running"

6.2 Towards fine-tuning based methods

Although this work primarily focuses on tune-free modifications to the self-attention mechanism, it is natural to ask how fine-tuning based methods might perform in a similar multiview setting. Below, we outline two commonly adopted strategies: parameter-efficient fine tuning and full model fine tuning. Both provide a complementary perspective to our proposed approach, but also highlight the trade-offs between efficiency, adaptability, and computational cost.

6.2.1 Parameter-Efficient Fine-Tuning (DreamBooth with LoRA)

DreamBooth introduced the idea of personalizing text to image diffusion models by conditioning on a few reference images and associating them with novel identifiers in the text prompt [14]. While highly effective, its original formulation requires updating a large number of parameters, which can be prohibitively expensive in multiview or multi-identity scenarios.

Recent work on Low-Rank Adaptation (LoRA) addresses this limitation by injecting low rank matrices into the attention layers, thereby enabling efficient fine-tuning without modifying the full weight matrices [15]. In practice, DreamBooth combined with LoRA has become a standard parameter-efficient approach to personalization.

In the context of multiview generation, such methods could allow the model to better adapt to the geometry and appearance of a specific subject across different viewpoints. However, the reliance on additional optimization introduces new challenges. Multiview consistency

often requires either multiple annotated viewpoints or carefully designed data augmentation strategies, which can reduce data efficiency. At the same time, fine-tuning on a limited set of views increases the risk of overfitting, causing the model to lose generalization and produce artifacts when asked to generate novel perspectives. Finally, there is the added computational burden: even with parameter-efficient approaches such as LoRA, the cumulative updates become significant when extending to multiple subjects or tasks.

By contrast, AR approach avoids these concerns by modifying the attention mechanism at inference time without additional training. Nonetheless, parameter efficient methods represent an important complementary direction for improving subject fidelity and multiview alignment.

6.2.2 Full Fine-Tuning

Full fine-tuning refers to updating all or most of the parameters of the diffusion backbone, such as the UNet or text encoder, for a target task. This approach offers maximum flexibility, since the model can fully adapt to the statistics of the training distribution, and in principle it could yield stronger geometric consistency across views by explicitly learning to bind appearance and structure under the multiview setting. However, these benefits come at significant cost [13]. Full fine-tuning is highly resource intensive, demanding substantial GPU memory and training time, which makes it impractical for large backbones such as Stable Diffusion 2.1 or SDXL. It also suffers from catastrophic forgetting, as adapting to a small subject-specific dataset often degrades performance on unrelated prompts, thereby reducing versatility. Moreover, scalability is limited, since every new subject or geometry in a multiview scenario requires repeating the entire fine-tuning process, which quickly becomes infeasible. While full fine-tuning may still be appropriate in specialized production environments, such as training a dedicated multiview generator for a fixed domain, our experiments suggest that tune-free alternatives like AR provide a more practical path. By operating at the architectural level without updating parameters, AR improves spatial consistency while retaining generality, offering a middle ground between fixed pretrained models and costly fine-tuned ones.

6.2.3 Other Directions in the Field

Mingdeng Cao, the author of MasaCtrl has been a leading figure in tune-free image editing methods. His recent works, such as InstructMove, focuses on fine-tuning for more controlled edits [3]. A persistent limitation in this space is that target images are often synthetically generated, constraining performance due to dataset quality issues. His work captures videos at different frames to create a real dataset in which pairs of edited images are not artificially generated, thereby improving realism and training signal.

While this approach is not yet applied to multiview image editing, the trend toward building custom, high-quality datasets is evident. In addition, prompt engineering for image editing has evolved: moving from “asking” the model to explicitly “instructing” it has been shown to improve edit fidelity [3]. These ideas can likewise be applied to multiview image editing.

6.3 Potential Applications

Beyond image generation, these methods have potential in a wide range of downstream tasks such as 3D reconstruction, mesh generation, and scene understanding. As mentioned before, multiview image editing assists in mesh editing due to more reliable keypoint extraction, and higher quality keypoints yield higher fidelity meshes. This directly benefits applications in character animation and virtual reality, where consistent multiview geometry is required for smooth rigging and deformation.

Another key use case lies in robotics and embodied AI, where multiview consistency can be leveraged for robust object manipulation and navigation. High fidelity reconstructions allow agents to reason about object geometry and grasp points, which are critical for tasks such as robotic grasping or human robot interaction.

In simulation and content creation, these methods could accelerate the pipeline of generating 3D assets for games, movies, and AR/VR applications. Artists can iteratively refine views of an object, while the system ensures geometric and stylistic consistency across all perspectives. This drastically reduces the manual workload in mesh sculpting or texture painting.

6.4 Conclusion and Acknowledgments

In summary, both MVDream 2.1 with AR and masking, and MVDream 1.5 with AR and IP-Adapter, produce strong generations. However, when balancing quality and computational efficiency, MVDream 1.5 emerges as the most practical choice, despite a modest trade-off in robustness.

This work underscores the importance of efficient attention mechanisms, balanced architectural design, and the interplay between runtime constraints and output quality. Future research should explore fine-tuning lightweight adapters, integrating higher-quality datasets, and developing attention mechanisms that can combine identity preservation and multiview consistency without sacrificing speed.

We thank the authors of MVDream and IP-Adapter for releasing their models, and the creators of MVAdapter and MasaCtrl for providing inspiration and foundational methods. We also acknowledge the use of asset data for the input images and the base repository <https://github.com/odeb1/project>, on which this work was built [11]. Special thanks to my friends and family for supporting me during my time at Oxford. Thanks for reading.

References

- [1] Y. Shi, P. Wang, J. Ye, M. Long, K. Li, X. Yang. *MVDream: Multi-view Diffusion for 3D Generation*, 2024. arXiv:2308.16512. <https://arxiv.org/abs/2308.16512>.
- [2] Z. Huang, Y.-C. Guo, H. Wang, R. Yi, L. Ma, Y.-P. Cao, L. Sheng. *MV-Adapter: Multi-view Consistent Image Generation Made Easy*, 2024. arXiv:2412.03632. <https://arxiv.org/abs/2412.03632>.
- [3] M. Cao, X. Zhang, Y. Zheng, Z. Xia. *Instruction-based Image Manipulation by Watching How Things Move*, 2024. arXiv:2412.12087. <https://arxiv.org/abs/2412.12087>.
- [4] B. Liu, C. Wang, T. Cao, K. Jia, J. Huang. *Towards Understanding Cross and Self-Attention in Stable Diffusion for Text-Guided Image Editing*, 2024. arXiv:2403.03431. <https://arxiv.org/abs/2403.03431>.
- [5] J. Xiang. *Color ControlNet (GitHub repository)*. https://github.com/jinxixiang/color_controlnet.
- [6] H. Ye, J. Zhang, S. Liu, X. Han, W. Yang. *IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models*, 2023. arXiv:2308.06721. <https://arxiv.org/abs/2308.06721>.
- [7] L. Zhang, A. Rao, M. Agrawala. *Adding Conditional Control to Text-to-Image Diffusion Models*, 2023. arXiv:2302.05543. <https://arxiv.org/abs/2302.05543>.
- [8] Q. Deng, O. Deb, A. Patel, C. Rupprecht, P. Torr, N. Trigoni, A. Markham. *Towards Multi-Modal Animal Pose Estimation: A Survey and In-Depth Analysis*, 2025. arXiv:2410.09312. <https://arxiv.org/abs/2410.09312>.
- [9] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer. *High-Resolution Image Synthesis with Latent Diffusion Models*. CVPR, 2022. <https://arxiv.org/abs/2112.10752>.

- [10] J. Song, C. Meng, S. Ermon. *Denoising Diffusion Implicit Models*, 2022. arXiv:2010.02502. <https://arxiv.org/abs/2010.02502>.
- [11] O. Deb. *Project repository (GitHub)*. <https://github.com/odeb1/project>.
- [12] M. Cao, X. Wang, Z. Qi, Y. Shan, X. Qie, Y. Zheng. *MasaCtrl: Tuning-Free Mutual Self-Attention Control for Consistent Image Synthesis and Editing*, 2023. arXiv:2304.08465. <https://arxiv.org/abs/2304.08465>.
- [13] Reece Shuttleworth, Jacob Andreas, Antonio Torralba, and Pratyusha Sharma. *LoRA vs Full Fine-tuning: An Illusion of Equivalence*. arXiv preprint arXiv:2410.21228, 2025. URL: <https://arxiv.org/abs/2410.21228>.
- [14] Nataniel Ruiz, Yuanzhen Li, Varun Jampani, Yael Pritch, Michael Rubinstein, and Kfir Aberman. *DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation*. arXiv preprint arXiv:2208.12242, 2023. URL: <https://arxiv.org/abs/2208.12242>.
- [15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv:2106.09685, 2021. URL: <https://arxiv.org/abs/2106.09685>.

A Reproducibility

The code was executed with two separate environments, one configured for MVDream (both 1.5 and 2.1), as well a separate Conda environment for MVAdapter. The yml files are included in the zip files, though the IP Adapter model weights and code need to be extracted from its Github repository at: <https://github.com/tencent-ailab/IP-Adapter/tree/main>.

Figure 1: DDIM Inversion

```
python mvdream/scripts/ddim_inversion.py  
  
--model_name "sd-v2.1-base-4view"  
  
--seed 2025  
  
--text "a tiger standing still"  
  
--animal_name "tiger_8_views_rest"  
  
--num_frames 16  
  
--step 50  
  
--camera_azim 135
```

Note: As long as the visualization is not commented, the cached latents are saved for each timestep after DDIM Inversion.

Figure 4: Interleaved Generation

```
program: mvdream/scripts/articulation.py  
  
args:  
  
--model_name "sd-v2.1-base-4view"  
  
--seed 2025  
  
--inversion_seed 2025  
  
--text "a tiger is sitting down"  
  
--animal_name "tiger_8_views_rest"  
  
--num_frames 16  
  
--step 50
```

```
--ddim_depth 25
```

Figure 7 (Default):

```
program: mvdream/scripts/articulation.py
```

```
args:
```

```
--model_name "sd-v2.1-base-4view"  
--seed 2025  
--inversion_seed 2025  
--text "a tiger is sitting down"  
--animal_name "tiger_8_views_rest"  
--num_frames 16  
--step 50  
--ddim_depth 25  
--use_vanilla_attention
```

Figure 7 (AR):

```
program: mvdream/scripts/articulation.py
```

```
args:
```

```
--model_name "sd-v2.1-base-4view"  
--seed 2025  
--inversion_seed 2025  
--text "a tiger is sitting down"  
--animal_name "tiger_8_views_rest"  
--num_frames 16  
--step 50  
--ddim_depth 25
```

Figure 9 (Masked Generation):

```
program: mvdream/scripts/articulation.py
```

```
args:
  --model_name "sd-v2.1-base-4view"
  --seed 2025
  --inversion_seed 2025
  --text "a tiger is sitting down"
  --source_prompt "a tiger is standing still"
  --animal_name "tiger_8_views_rest"
  --num_frames 16
  --step 50
  --ddim_depth 25
  --threshold 0.63
```

Figure 12 (FPE mix with RA):

program: mvdream/scripts/articulation.py

```
args:
  --model_name "sd-v2.1-base-4view"
  --seed 2025
  --inversion_seed 2025
  --text "a tiger is sitting down"
  --source_prompt "a tiger is standing still"
  --animal_name "tiger_8_views_rest"
  --num_frames 16
  --step 50
  --ddim_depth 25
  --threshold 0.63
  --timestep_End_FPE 14
  --layer_RSA_Switch 5
  --bottom_layer_FPE 0
  --top_Layer_FPE 9999
```

Figure 18: MVAdapter with Scribble Controlnet

```
program: MV-Adapter/scripts/inference_scribble2mv_sd21.py
args:
  --text "a tiger sitting down"
  --controlnet_images
    MVDream/assets/renderings/"tiger_8_views_rest"/tiger_045.png
    MVDream/assets/renderings/"tiger_8_views_rest"/tiger_090.png
    MVDream/assets/renderings/"tiger_8_views_rest"/tiger_135.png
    MVDream/assets/renderings/"tiger_8_views_rest"/tiger_180.png
    MVDream/assets/renderings/"tiger_8_views_rest"/tiger_225.png
    MVDream/assets/renderings/"tiger_8_views_rest"/tiger_270.png
    MVDream/assets/renderings/"tiger_8_views_rest"/tiger_315.png
    MVDream/assets/renderings/"tiger_8_views_rest"/tiger_360.png
  --guidance_scale 5.0
  --controlnet_conditioning_scale 0.7
```

Figure 21b: MVAdapter Elephant with its trunk down

```
program: MV-Adapter/scripts/ablation.py
args:
  --text "an elephant with its trunk down"
  --num_inference_steps 50
  --image_dir "MVDream/assets/renderings/elephant_8_views"
  --output_dir "/work/davidfan/MVAdapter_results/inversion_outputs"
  --num_views 8
  --size 512
  --step 50
```

```

--start_step 25
--scale 5.0
--eta 0.0
--interleaved
--seed 42
--device cuda
--fp16
--visualize_depth -1
--mv_lower_range 0.5
--mv_upper_range 0.5
--rsa_lower_range 0.5
--rsa_upper_range 0.5
--cur_sa 1

```

Figure 25: Default MVDream 1.5

```

python scripts/t2i.py --model_name "sd-v1.5-4view"
--text "a tiger sitting down"

```

Figure 26: Default MVDream 2.1

```

python scripts/t2i.py --model_name "sd-v2.1-base-4view"
--text "a tiger sitting down"

```

Note: This command uses the default implementation from the official MVDream repository at <https://github.com/bytedance/MVDream>.

Figure 30c: Hummingbird with IP Adapter

```

program: mvdream/scripts/articulation.py
args:
    --model_name "sd-v1.5-4view"

```

```
--seed 2030
--inversion_seed 2030
--text "a hummingbird sitting down"
--animal_name "humming_8_views"
--num_frames 16
--step 50
--ddim_depth 25
--use_ip_adapter
```

Figure 33: Elephant with IP Adapter

program: mvdream/scripts/articulation.py

args:

```
--model_name "sd-v1.5-4view"
--text "an elephant lowering its trunk"
--seed 2025
--inversion_seed 2025
--animal_name "elephant_8_views"
--num_frames 16
--step 50
--ddim_depth 25
--use_ip_adapter
```

Figure 34: Rabbit with IP Adapter

program: mvdream/scripts/articulation.py

args:

```
--model_name "sd-v1.5-4view"
--seed 2032
--inversion_seed 2032
```

```
--text "a rabbit sitting down"
--animal_name "rabbit"
--num_frames 16
--step 50
--ddim_depth 25
--use_ip_adapter
```

Figure 35: Alpaca with IP Adapter

program: mvdream/scripts/articulation.py

args:

```
--model_name "sd-v1.5-4view"
--seed 2716
--inversion_seed 2716
--text "an alpaca running"
--animal_name "alpaca"
--num_frames 16
--step 50
--ddim_depth 25
--use_ip_adapter
```